

Assignment3 report

Haoyu

2026/02/22

1 Introduction

This lab reports explores three foundational machine learning techniques:

- Sparse Autoencoder for unsupervised representation learning
- Neural Language Model for sequence modeling
- Denoising Diffusion Probabilistic Model for generative image synthesis

The objective is to understand both the theoretical principles and the implementation details of these algorithms.

2 Assignment 3A: Sparse Autoencoder

2.1 Objective

The objective of this assignment was to implement a sparse autoencoder capable of learning meaningful representations from natural images. The model aims to reconstruct the input while enforcing sparsity constraints on hidden unit activations. This experiment illustrates how unsupervised learning can extract structured features without labeled data.

2.2 Methods

2.2.1 Training Data Generation

Training samples were generated by randomly extracting 8×8 patches from the provided dataset of natural images. Each patch was reshaped into a 64-dimensional vector. A total of 10,000 patches were sampled to construct the training dataset.

2.2.2 Sparse Autoencoder Model

The sparse autoencoder consisted of:

- Input layer: 64 units
- Hidden layer: 25 units
- Output layer: 64 units

The sigmoid activation function was used:

$$f(z) = \frac{1}{1 + e^{-z}}$$

The cost function included three components:

1. Reconstruction error
2. Weight decay regularization
3. Sparsity penalty (KL divergence)

Parameters were optimized using the L-BFGS algorithm.

2.2.3 Gradient Checking

To verify the correctness of the analytical gradients, numerical gradient checking was implemented using finite differences with $\epsilon = 10^{-4}$. The comparison confirmed close agreement between numerical and analytical gradients.

2.3 Results

After training, the learned weights were visualized using the provided display function.

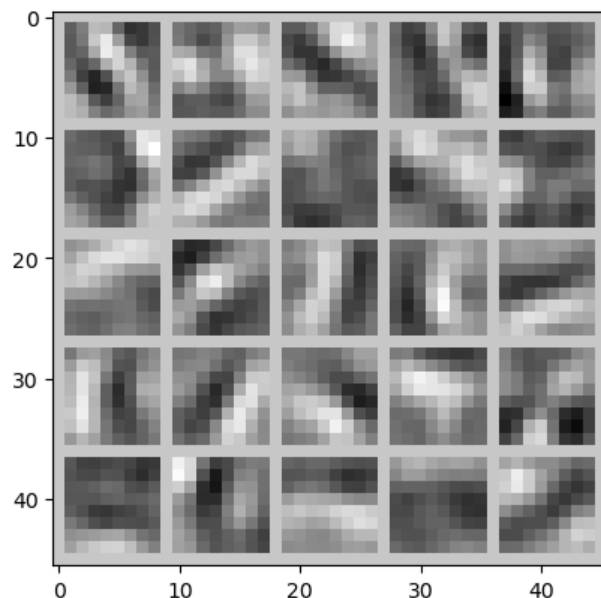


Figure 1: Visualization of Learned Features

The learned filters exhibit structured patterns resembling edges and oriented lines. These features are characteristic of natural images and indicate successful representation learning.

2.4 Discussion

The results demonstrate that the sparse autoencoder successfully learned meaningful low-level features. The emergence of edge-like structures aligns with expectations that edges form a basis for representing natural images.

Several implementation challenges were encountered:

- Correctly reshaping parameters between vectorized and matrix forms
- Ensuring gradient correctness through debugging

2.5 Conclusion

This assignment illustrates the effectiveness of sparse autoencoders in unsupervised feature learning. The model successfully captured fundamental image structures, demonstrating how regularization and sparsity shape learned representations.

3 Assignment 3B: Neural Language Model

3.1 Objective

The objective of this assignment was to train and evaluate a Transformer-based language model on the Shakespeare dataset. The emphasis of this experiment is understanding the Transformer architecture, attention mechanisms, and generative behavior.

3.2 Methods

3.2.1 Transformer Architecture

The implemented model follows a Transformer-based autoregressive design. Unlike recurrent neural networks, the Transformer models dependencies using self-attention mechanisms.

The model consists of:

- Token embeddings
- Positional embeddings
- Stacked Transformer blocks
- Multi-head self-attention layers
- Feed-forward networks
- Output projection layer

Self-attention is defined as:

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

This mechanism enables dynamic context-dependent interactions between tokens.

3.3 Results

3.3.1 Training and Evaluation

Model parameters were selected to balance representational capacity and computational feasibility. The embedding dimension allows sufficient semantic representation, while multiple attention heads capture diverse token relationships. Training produced the following losses:

- Training Loss: 1.153
- Validation Loss: 0.989

The reduction in training loss indicates successful learning of token distributions. The validation loss being slightly lower suggests stable generalization behavior.

3.3.2 Generated Text Samples

Example generated output:

ROMEO:

Many thing on this fair cannot better the crown'd wife
Of well; and, let it see fight.

MERCUTIO:

Well, I'll tell your lordship: holy succession,
That you may seem for this conduct to be both:
This is the way of the corn of him.

BENVOLIO:

I am sain

The generated text exhibits several characteristics of Shakespeare-style language:

- Dialogue structure with speaker labels
- Grammatical consistency to a certain extent
- typical Shakespeare style

Although semantic inconsistencies appear, the stylistic structure is preserved.

3.4 Discussion and Conclusion

Imperfections in generated text primarily reflect limited training duration rather than architectural deficiencies.

This assignment provided deeper insight into:

- Self-attention dynamics
- Probabilistic language modeling
- Differences from recurrent models
- Sampling-driven variability

The Transformer language model successfully captures basic statistical properties of the Tiny Shakespeare dataset. More importantly, the experiment reinforces my conceptual understanding of attention-based sequence modeling.

4 Assignment 3C: Diffusion Model (DDPM)

4.1 Objective

The objective of this assignment was to implement and analyze a Denoising Diffusion Probabilistic Model (DDPM) trained on the MNIST dataset. The primary focus of this experiment is understanding diffusion-based generative modeling, including noise scheduling, denoising dynamics, and probabilistic sampling.

4.2 Methods

4.2.1 Diffusion Framework

Unlike traditional generative models that learn a direct mapping from latent space to data space, diffusion models define generation as a two-stage stochastic process:

1. Forward diffusion
2. Reverse diffusion

The forward process gradually corrupts the data:

$$x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$$

where $\epsilon \sim \mathcal{N}(0, I)$.

The reverse process reconstructs samples via learned noise prediction:

$$x_{t-1} = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_\theta(x_t, t) \right) + \sigma_t z$$

4.2.2 Key Components

The implemented model includes:

- Noise scheduler with linear variance schedule
- Sinusoidal timestep embeddings
- U-Net noise prediction network
- Mean squared error training objective

The model learns to predict the noise component:

$$\mathcal{L} = \mathbb{E}_{t, x_0, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t)\|^2]$$

4.2.3 Why Diffusion Models Are Stable

Diffusion models convert a complex generation problem into a sequence of local denoising steps. Instead of adversarial training, the objective is simple noise regression, which improves stability.

The forward process defines a smooth degradation trajectory that maps data distributions toward Gaussian noise. This creates a well-behaved learning target.

The reverse process approximates the score function of the data distribution. Each step refines structure by removing estimated noise components.

4.3 Results

4.3.1 Forward Diffusion Process

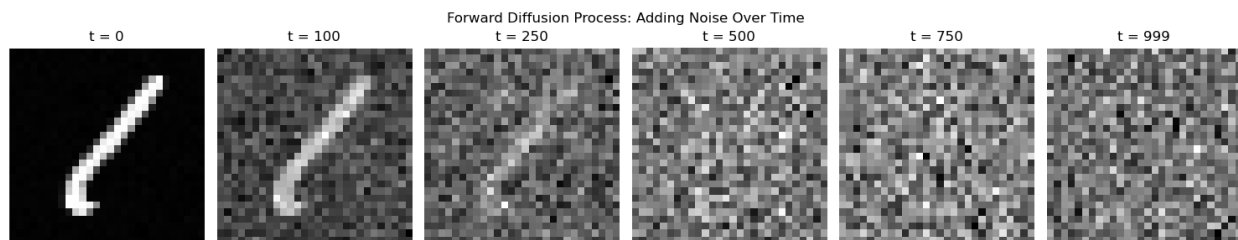


Figure 2: Forward Diffusion: Progressive Noise Injection

As timestep increases, structured information gradually disappears. Early stages still preserve digit structure, while later stages are more like pure Gaussian noise. This validates a correct noise scheduling.

4.3.2 Reverse Diffusion Process

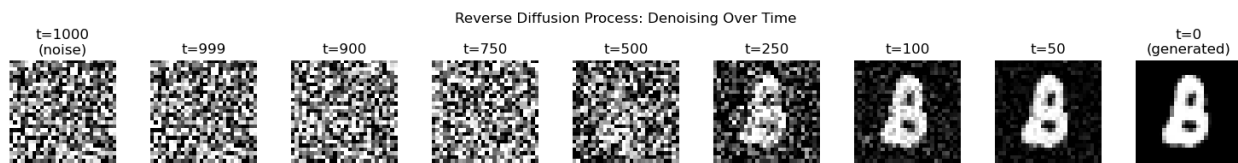


Figure 3: Reverse Diffusion: Progressive Denoising

The reverse trajectory illustrates gradual structure emergence. Starting from noise, recognizable digit patterns appear as the model iteratively removes noise.

4.3.3 Generated Samples

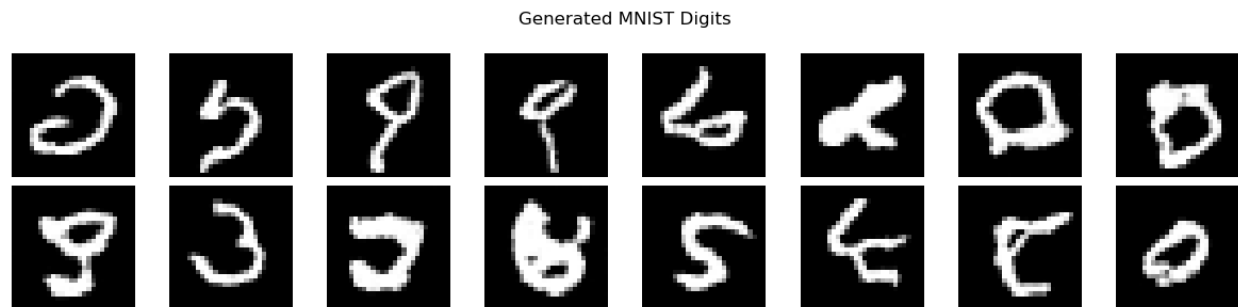


Figure 4: Generated MNIST Digits

The generated samples display meaningful digit-like structures consistent with MNIST statistics. While some meaningless pattern also exists. Variability across samples reflects stochastic sampling.

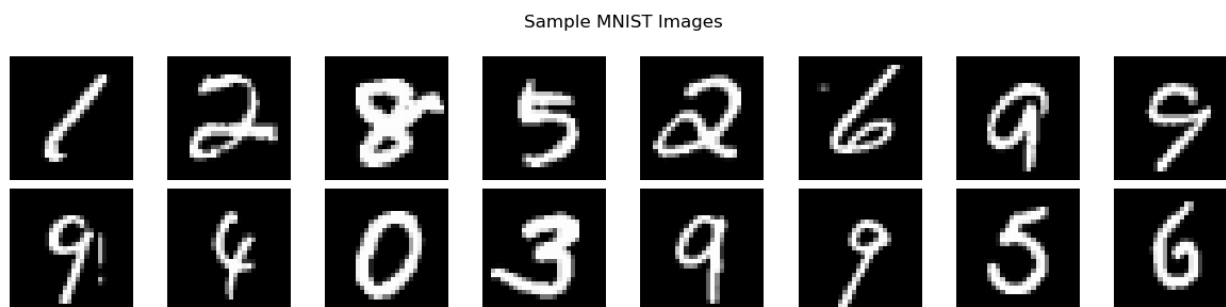


Figure 5: Real MNIST Samples

Comparison indicates that the model captures key visual characteristics such as the geometry and spatial layout.

4.3.4 Training Dynamics

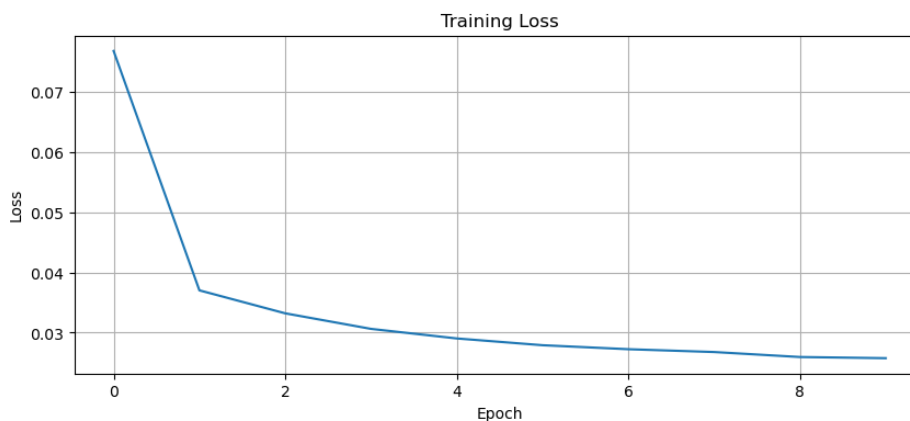


Figure 6: Training Loss Curve

The loss decreases steadily, demonstrating stable optimization. The rapid early reduction corresponds to learning coarse noise structure, followed by slower refinement.

4.3.5 Generated vs Real Samples

Similarity between generated and real digits suggests successful modeling of dataset statistics. Minor distortions reflect limited training duration rather than model failure.

4.3.6 Evaluation

The experiment demonstrates:

- Correct forward diffusion dynamics
- Stable reverse denoising behavior
- Successful sample synthesis
- Stable training convergence

4.4 Discussion and Conclusion

This assignment provided insight into:

- Probabilistic generative modeling
- Noise-conditioned learning
- Differences between direct and iterative generation
- The role of stochastic processes in synthesis

Diffusion models illustrate how complex distributions can be learned through incremental refinement rather than direct mapping.

The implemented DDPM successfully models MNIST data through learned denoising dynamics. More importantly, this experiment reinforces conceptual understanding of diffusion-based generative modeling, highlighting its stability and interpretability.

5 Conclusion

This assignment highlights:

- The importance of regularization and gradients
- The role of sampling strategies
- Probabilistic modeling advantages

Understanding these methods lays the foundation for modern AI techniques.